ExoticCare — Final Project Documentation

1. System Architecture Overview

1.1 Architecture Approach

The approach for this project is a layered approach which has different layers and branches off into layers like the presentation, business, data, and database layers. This not only makes it easier for me, but it keeps things more together and have each responsibility organized. This approach would also make the project much easier to maintain and modify.

1.2 Presentation Layer

The Presentation Layer will be able to provide the user interface for ExoticCare which will be done by using Java Swing. Users will also be able to interact with this layer to create and manage their exotic pet profiles, view pet information, manage feeding schedules, create care tasks, and view reminders.

The Presentation Layer will be responsible for showing all the information to the user and collecting the user's input as well. It will also not directly communicate with the database at all.

1.3 Business Logic Layer

The Business Logic Layer will show the application's main rules and functions of how they work. This layer will also show the requests from the user interface and determine how ExoticCare should handle them going forward.

The Business Logic Layer will also help function parts such as creating and updating pet profiles, while also recording feeding schedules, managing care tasks, and making sure which reminders should be displayed to the user.

1.4 Data Access Layer

The is layer is the Data Access layer, and this is where application and the database will communicate with one another. The Java database connectivity is what will be used to make sure that Java and the database communicate properly.

The Data Access layer will also take care of responibilities like saving information and also helping to take care of many tasks to help users save their information.

1.5 Database Layer

ExoticCare will use SQLite as its database. The database will help hold important information that is needed in the application like, including the user information, the users exotic pet profiles, feeding schedules, care tasks, and any other reminders that might be needed.

SQLite was chosen to be the database because it's very easy to manage, and it also doesn't require any separate database server, making it a good choice for ExoticCare .

1.6 Architecture Data Flow

This is how the data flow for ExoticCare will be:

User → Presentation Layer → Business Logic Layer → Data Access Layer → SQLite Database

When the process is done, it would then work in the opposite direction:

SQLite Database → Data Access Layer → Business Logic Layer → Presentation Layer → User

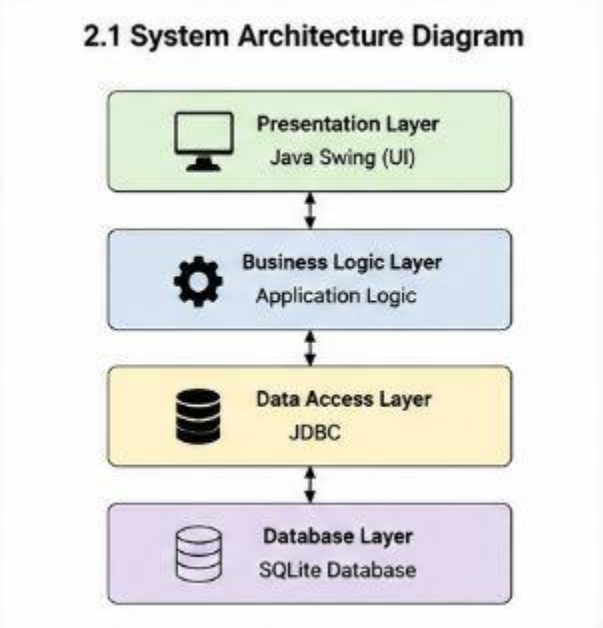
Each event being separated does allow each layer to focus on a specific responsibility and helps support the application's object-oriented and to be able to keep it's design.

1.7 Technology Stack

The technology stack for ExoticCare is:

- Java: primary programming language
- Java Swing: graphical user interface
- SQLite: database management
- JDBC: database connectivity
- JUnit: software testing
- Git/GitHub: version control

1.8 System Architecture Diagram



The system architecture follows the flow:

User → Java Swing Presentation Layer → Business Logic Layer → JDBC/Data Access Layer → SQLite Database

2. UML Class Diagram

2.1 Class Diagram Overview

The UML Class Diagram shows how the main classes will be used to develop the application, and the diagram shows the relationships between the user, exotic pets, care tasks, and reminders.

The main classes are the user, ExoticPet, CareTask, and reminders. These classes help represent the main objects that are needed to manage all the pet information and organize care responsibilities.

User

This class here shows how a person uses the ExoticCare application. A user could then manage information for one or more exotic pets that they own or know.

Primary attributes:

- userId
- name
- email

Primary responsibilities:

- Maintain user information
- Manage associated exotic pet profiles

ExoticPet

The ExoticPet class represents an exotic animal managed by the user. Each pet profile stores basic information about the animal.

Primary attributes:

- petId
- name
- species
- breed/type
- age

Primary responsibilities:

- Store pet information
- Allow pet information to be viewed and updated
- Associate care tasks with a specific pet

CareTask

The CareTask class represents a basic care responsibility for an exotic pet. Some of the tasks that are included are feeding, cleaning, bathing, habitat maintenance, or other routine activities.

Primary attributes:

- taskId
- taskName
- description
- dueDate
- frequency
- completed

Primary responsibilities:

- Store care-task information
- Associate tasks with a pet
- Track whether a task has been completed
- Store scheduled care information

Reminder

The Reminder class represents a notification associated with a scheduled care task. It's important to have these reminders because they can help users maintain or keep track of things that they need.

Primary attributes:

- reminderId
- reminderDate
- status

Primary responsibilities:

- Associate reminders with care tasks
- Store reminder dates
- Track reminder status

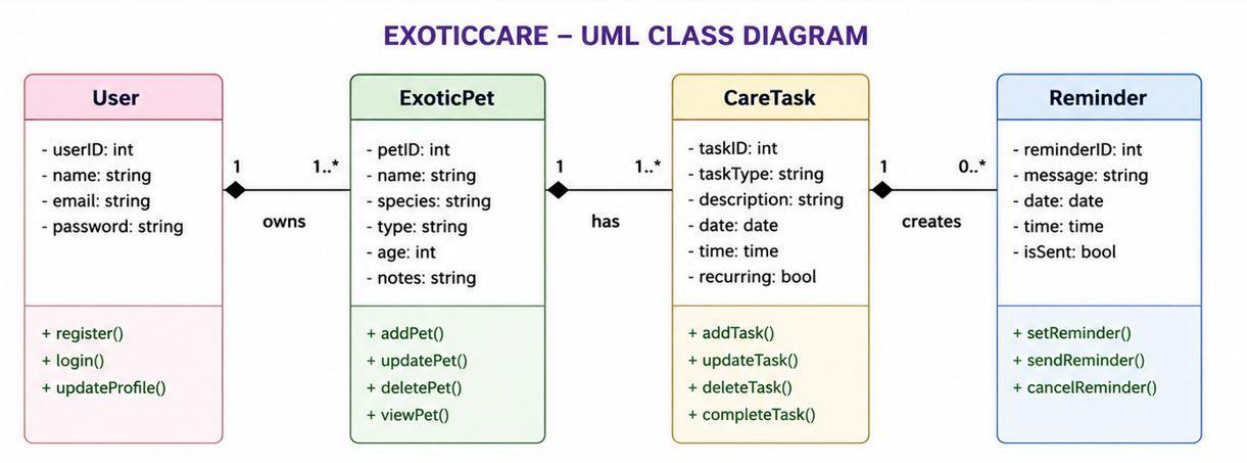
2.2 Class Relationships

The classes are connected through the following relationships:

- A User can manage multiple ExoticPet profiles.
- Each ExoticPet belongs to a user.
- An ExoticPet can have multiple CareTask objects.
- Each CareTask is associated with an exotic pet.
- A CareTask can have one or more Reminder objects.
- Each Reminder is associated with a care task.

These relationships allow ExoticCare to connect a user's pets with the care responsibilities and reminders associated with each pet.

2.3 Class Diagram



The UML Class Diagram from the project proposal will be used as the foundation for the final system design. This will follow the classes and relationships represented in the diagram.

2.3 UML Sequence Diagram

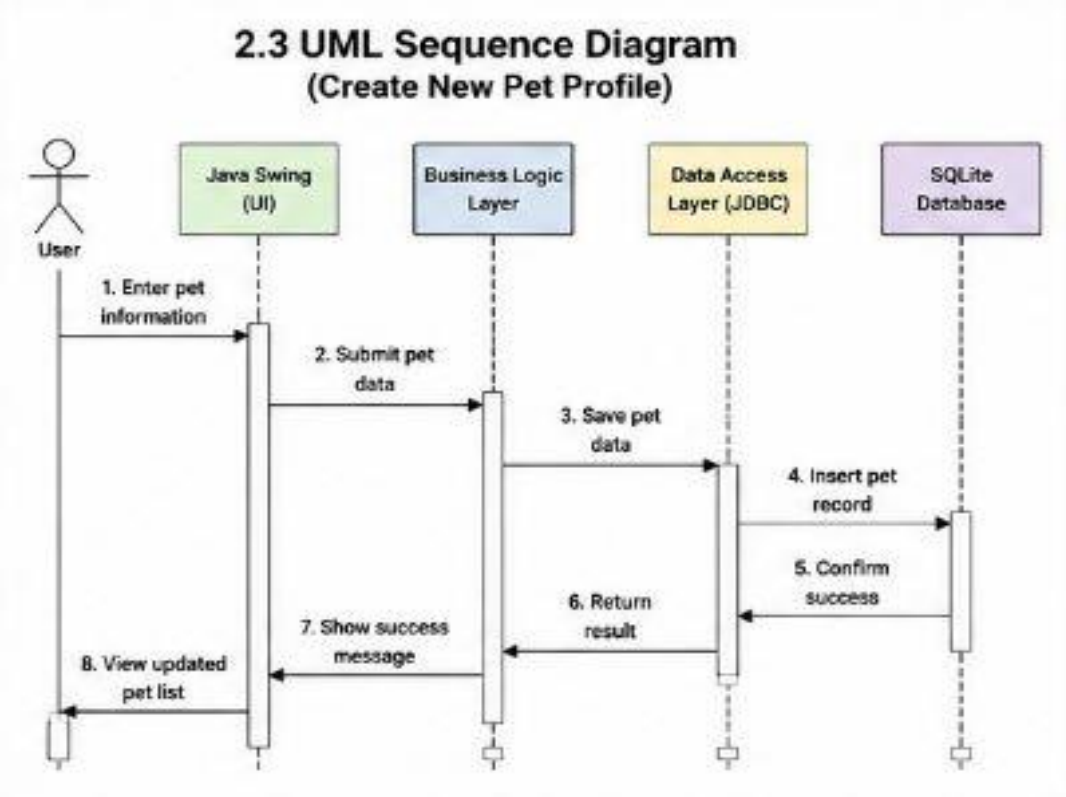
The UML Diagram here shows how all the different components of ExoticCare will interact when a user creates a new exotic pet profile. It also helps show how the information moves from the user interface through the business logic and data access layers before being stored in the SQLite database.

- Sequence of Events

1. The user selects the option to create a new pet profile.
2. The system displays the pet profile form.
3. The user enters the pet's name, species, breed, and age.
4. The application sends the information to the business logic layer.
5. The business logic layer validates the information.
6. The information that is correct is then sent to the data access layer.
7. The data access layer communicates with the SQLite database.
8. SQLite stores the new pet profile.

9. The database returns a successful result.
10. The application displays a confirmation message to the user.

UML Sequence Diagram Image



2.4 UML Deployment Diagram

The Deployment Diagram shows how the physical environment in which the ExoticCare application will work in. The application will also run locally on the user's computer using Java and Java Swing. The SQLite database will also be stored locally and will be accessed by the application through JDBC.

The deployment design consists of a user's computer containing the ExoticCare application and its database. The user uses the application through the Java Swing user interface. The application also communicates with the SQLite database which goes through the data access layer which then stores and retrieves information.

This deployment is easier because it also helps keep the application and database work on the same device, which makes it much easier and allows ExoticCare to operate without needing a whole separate database server.

Deployment Components

The deployment environment will contain the following components:

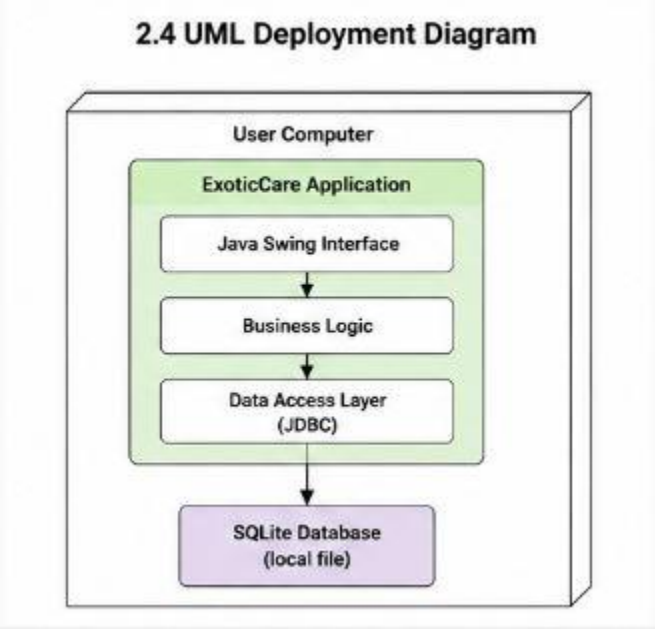
1. User Device
    a. The computer used by the ExoticCare user.
    b. Provides the environment where the application runs.
2. ExoticCare Application
    a. Developed using Java.
    b. Provides the Java Swing graphical user interface.
    c. Contains the business logic and data access components.
3. JDBC
    a. Provides communication between the Java application and SQLite.
    b. Allows the application to send database requests and retrieve stored information.
4. SQLite Database
    a. Stores user and pet information.
    b. Stores feeding schedules, care tasks, and reminders.
    c. Runs locally on the user's computer.

Deployment Flow

User → Java Swing Application → Business Logic → JDBC → SQLite Database

This is useful because when any information needs to be retrieved, the database will return any of the requested information through JDBC to the application, where it is then later processed and displayed to the user.

Deployment Diagram



2.5 Database Schema

This project will use the SQLite database to store and manage all the application information. The database will also allow the application to save the user's pet profiles, feeding schedules, care tasks, and reminders so that information will stay available. They will also stay available even when the application is closed and reopened.

The database will be used by the Java application, and the data access layer will take care of communication with the database, while the business logic layer will take care of how the information is used and stored by the application.

2.5.1 Database Tables

The ExoticCare database will contain the following primary tables:

Users Table

The Users table stores any of the information about people who use the ExoticCare application.

Field	Data Type	Description
user_id	INTEGER	Unique identifier for the user
name	TEXT	User's name
email	TEXT	User's email address

Pets Table

The Pets table stores the basic information associated with each exotic pet.

Field	Data Type	Description
pet_id	INTEGER	Unique identifier for the pet
user_id	INTEGER	Identifies the owner of the pet
name	TEXT	Name of the pet
species	TEXT	Species of the pet
breed_type	TEXT	Breed or type of pet
age	INTEGER	Age of the pet

Feeding Schedules Table

The Feeding Schedules table takes care of the information about anything being planned feeding routines for each pet.

Field	Data Type	Description
feeding_id	INTEGER	Unique identifier for the feeding schedule
pet_id	INTEGER	Identifies the associated pet
feeding_time	TEXT	Scheduled feeding time
frequency	TEXT	How often the pet should be fed
food	TEXT	Type of food to be provided

Care Tasks Table

The Care Tasks table stores basic care responsibilities associated with an exotic pet.

Field	Data Type	Description
task_id	INTEGER	Unique identifier for the care task
pet_id	INTEGER	Identifies the associated pet
task_name	TEXT	Name of the care task
description	TEXT	Description of the task
due_date	TEXT	Date the task is due
frequency	TEXT	How often the task should occur
completed	INTEGER	Indicates whether the task has been completed

Reminders Table

The Reminders table stores reminders associated with scheduled care activities.

Field	Data Type	Description
reminder_id	INTEGER	Unique identifier for the reminder
task_id	INTEGER	Identifies the associated care task
reminder_date	TEXT	Date of the reminder
status	TEXT	Current status of the reminder

2.5.2 Database Relationships

The tables will be connected through primary and foreign keys.

- A User can have multiple Pets.
- Each Pet is associated with one User.
- A Pet can have multiple Feeding Schedules.
- A Pet can have multiple Care Tasks.
- A Care Task can have multiple Reminders.

The relationships will allow ExoticCare to connect to each pet with its feeding schedule, care responsibilities, and reminders while keeping the stored information organized.

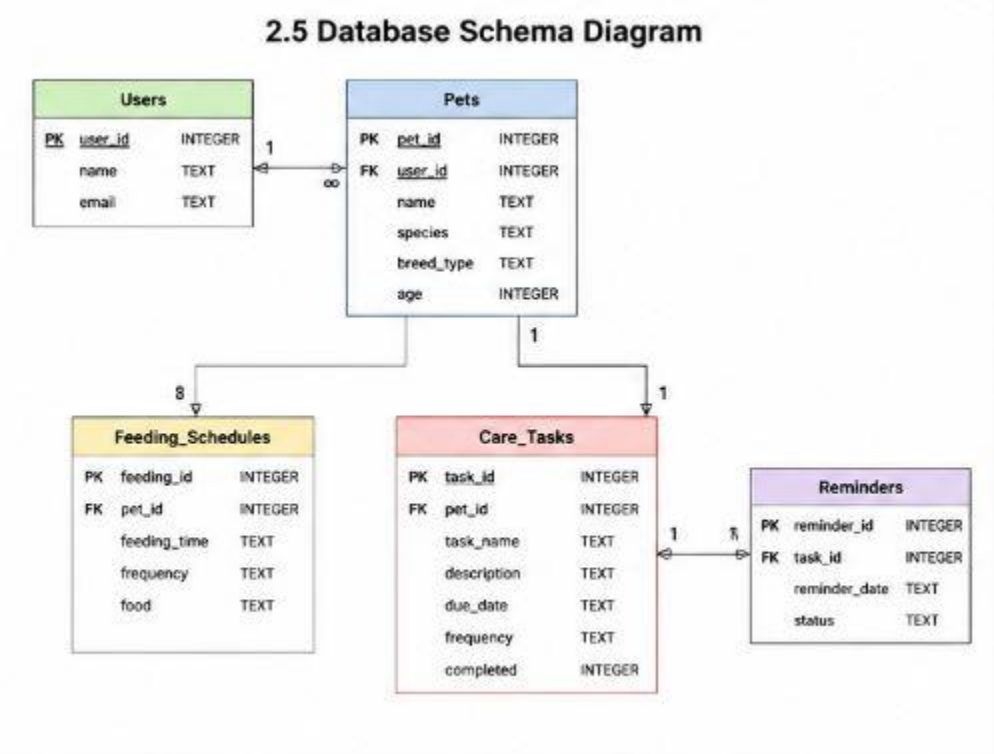
2.5.3 Database Structure

The overall relationship between the tables can be represented as:

Users
↓
Pets
↓
Feeding Schedules

Pets
↓
Care Tasks
↓
Reminders

2.5.4 Database Schema Diagram



2.6 User Interface Mockups

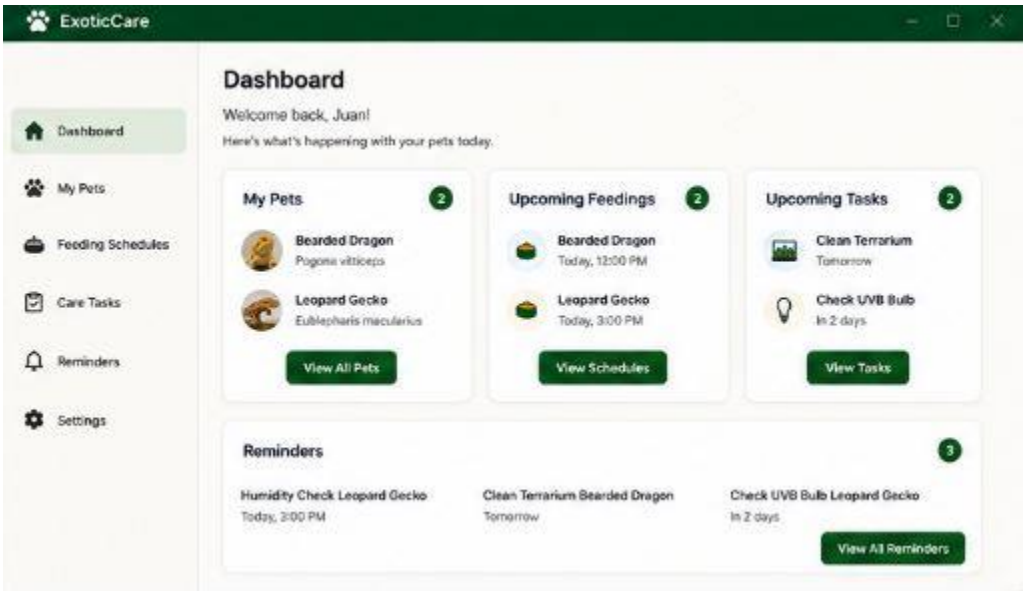
The main user interface for ExoticCare will be made to be simple and easy to use and navigate for users who want to manage their exotic pets and their care routines simple and have everything there for them. This user interface will focus on being easy to use and manage through the application and will have all reminders, schedules and other information on their pets right away.

The project will also use Java Swing to create the user interface. The design will use clear navigation and have well organized sections so that users can easily find and update their pet information.

2.6.1 Main Dashboard

The main dashboard for the project here will show all the tabs on the application, and it would give users access to their reminders and their pet profiles so that they can see all the things at once or look into the different tabs on their own.

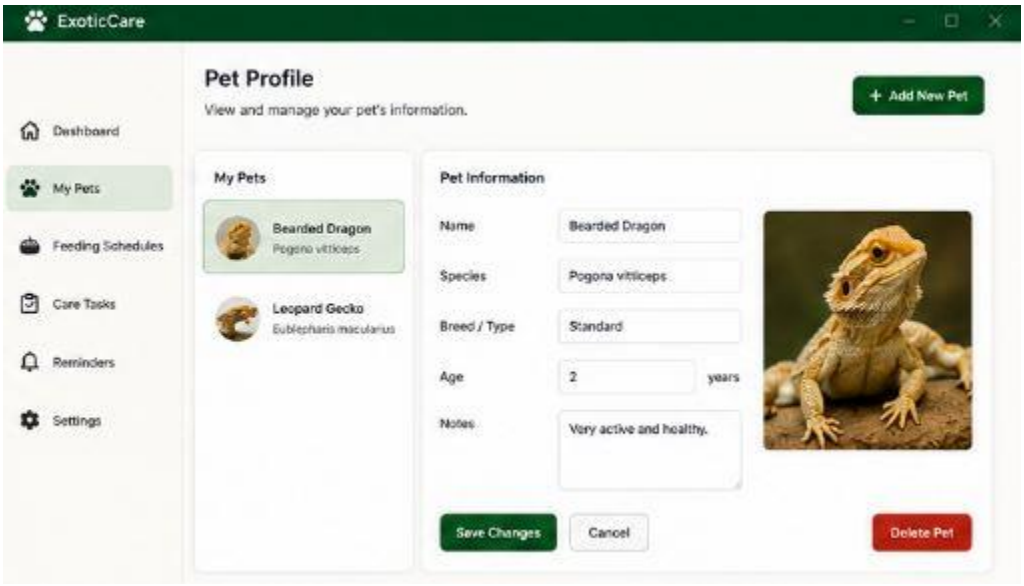
The dashboard will also have many navigation options for managing pets, feeding schedules, care tasks, and reminders.



2.6.2 Pet Profile Screen

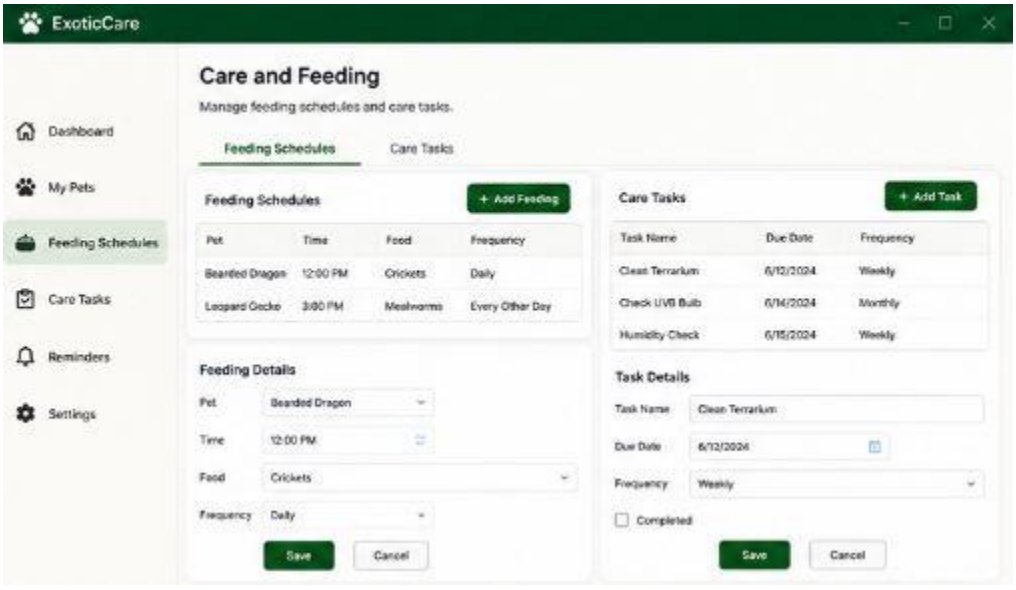
The Pet Profile page will allow show and help users to create, view, update, and delete exotic pet profiles. Users will be able to enter information as well asl update their information such as the pet's name, species, breed, and age.

The screen will provide controls for saving changes and managing existing pet information.



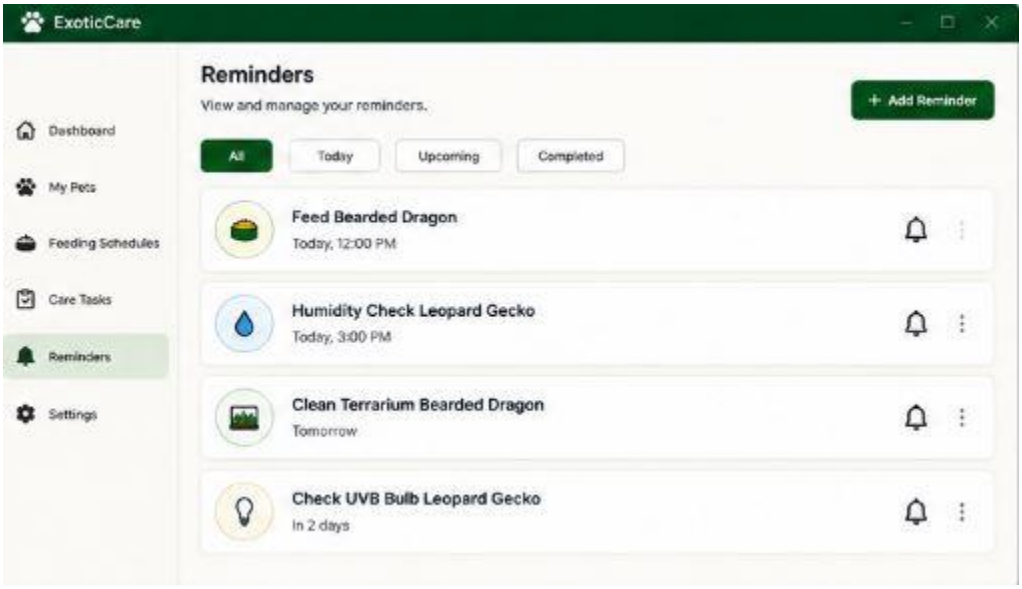
2.6.3 Care and Feeding Screen

The Care and Feeding screen will allow users to create and manage feeding schedules and basic care tasks. Users will be able to enter the type of task, scheduled time, or due date, frequency, and other relevant information.



2.6.4 Reminder Screen

The Reminder page will show any upcoming care activities and reminders. This will help the users to identify any of the tasks that need to be completed and keep track of their pets' regular care routines.



2.6.5 Interface Design Goals

The primary goals of the ExoticCare interface are:

- Keep navigation simple and easy to understand.
- Make pet information easy to access.
- Clearly display upcoming care activities.
- Allow users to quickly create and update information.
- Provide clear feedback when information is saved or changed.
- Maintain a consistent layout throughout the application.

Phase 3: Implementation and Testing

3.1 Application Implementation

ExoticCare was made as a Java desktop application that used the Java Swing to help make the user interface. The application also follows the layered architecture that was described during the design phase, and the presentation layer also contains the user interface, the service layer which

also handles the application logic and input validation, and the data access layer uses JDBC to communicate with the SQLite database.

The final version of the application has a fully functioning dashboard, pet profile management, feeding schedules, care tasks, and reminders as was planned and designed in the first process. The users can also create new, views, updates, and delete pet information. Feeding schedules and care tasks can also be associated with individual pets, while reminders can be connected to care tasks. Information entered into the application is stored in the SQLite database so that it remains available after the application is closed and reopened.

The final interface uses a green, white, and black color scheme with sidebar navigation between the major features of the application. The interface was updated during development to more closely match the UI mockups created during the design phase.

3.2 Version Control

ExoticCare uses Git and GitHub for version control. The project repository contains the Java source code, Maven configuration, application documentation, and testing files. Version control is used to track changes made throughout development and maintain an organized copy of the project.

3.3 Software Testing

Testing was performed to verify that the main features of ExoticCare operate correctly and that invalid information is handled appropriately. The testing process included unit testing, integration testing, system testing, and basic performance testing.

JUnit 5 was used for unit testing of the service layer. Three unit tests were executed to verify important input validation. These tests checked that a negative pet age is rejected, a blank pet name is rejected, and a feeding schedule cannot be created without required food information.

All three of the JUnit tests were passed successfully, and the final Maven test results reported 3 tests run, 0 failures, 0 errors, and 0 skipped tests, and the project completed with a BUILD SUCCESS result.

Integration and system testing were also performed manually through the Swing interface. The application was tested by adding, updating, and deleting pet profiles, creating feeding schedules and care tasks, marking care tasks as complete, and creating reminders. The application was restarted to verify that information stored in SQLite remained available.

Basic performance testing was performed during normal use of the application. Navigation between screens and the loading of stored records occurred within a few seconds without noticeable delays.

3.4 Code Review

The ExoticCare codebase was reviewed for organization, naming, validation, database access, and consistency with the planned layered architecture. The review included the model classes, service layer, DAO classes, Swing user interface, and SQLite persistence.

This project breaks down and separates the user interfaces, business logics, data access, and model classes into all separate packages to have their own responsibilities. This type of organization makes the responsibilities for the packages makes it much more easier to understand and maintain the application. The database that was used was the JDBC which was prepared with some of the resources that were put in place to manage the database operations and all their connections.

During the final development of the project, the user interface was updated many times to match the original UI mockups at were mocked at the first design. The sidebar navigation and an updated dashboard were added, and the pet, feeding schedule, care task, and reminder screens were updated. The application structure and database were also kept consistent with the final system design as well as the style and color of the whole application.

Some possible future improvements were identified during the review. Unit testing could be expanded to cover additional care tasks and reminder operations. Date fields could also include stricter date validation, and the reminder system could eventually automatically identify upcoming or overdue reminders.

Overall, the code review found that ExoticCare follows the planned layered architecture and supports the primary functionality established during the design phase.

Phase 4: Final Project Report

4.1 Development Process

ExoticCare was broken down and worked in multiple phases that first started with planning and design, which was then worked on by implementation and testing. The main UML diagrams and UI mockups were also created before the application was coded and developed. Java Swing was used for the interface, SQLite was used for data storage, and JDBC which was connected to the application to the database.

The application was built and tested feature by feature to make sure that everything worked properly and were no errors in the end. I also changed some parts to the interface to make things look a bit cleaner, but I kept the original design that I had when sketching the original.

4.2 Challenges Faced

Challenges that I faced during the coding process of the project were when I was setting ip the databases and trying to connect the drivers so that the databases could actually work and save information even after the application was closed. Another challenge that wasn't too bad but still was difficult was updating the Swing interface to match the UI mockups.

Setting up Maven for testing also requires additional configuration. After it was set up, all three JUnit tests passed successfully with no failures or errors.

4.3 Lessons Learned

This project has for sure helped me understand how the planning, development, databases, and testing work together in a complete software project. I also learned more about layered architecture, Java Swing, SQLite, JDBC, and JUnit testing. This also made me understand that it's important to document and make sure that doing things phase by phase is very organized and much more manageable in the end.

4.4 Conclusion

In the end, it was fun to document and create ExoticCare. Seeing it fully developed and functioning application was very accomplishing for me. Finishing it and making for managing and keeping track of exotic pet information, which is what it was planned to be, was very fun. Users can manage their pet profiles, feed schedules, care tasks, and reminders. The final application follows the original system design and stores information using SQLite. This was my biggest project to date, and I had fun through it.